

A MULTI-MODAL ADAPTIVE MICROTUNING ARCHITECTURE: BRIDGING MTS, MPE, AND MIDI 2.0 FOR STRUCK-STRING JUST INTONATION

Rui Su¹, Joseph Timoney¹, Damián Keller²

¹

¹Maynooth University, Maynooth, Ireland

²Ubimus

ABSTRACT

Current implementations of Real-Time Just Intonation (JI) are faced with a conundrum of precision versus compatibility. Tuning within MIDI is supported by the MIDI Tuning Standard (MTS) and pitch bend-based methods. MTS features good resolution but suffers from limited hardware support. Pitch bend-based methods provide ample compatibility but not enough granular resolution for microtonal tuning. This paper outlines an adaptive, multi-stage tuning architecture that extends the capabilities of the Web MIDI API and solves both of these problems. Within ubimus research agenda for struck string interactions, this proposed system utilizes a hierarchical fall back strategy: (1) MTS for scale-based tuning, (2) An MPE mode with precision optimized for locking the sensitivity of pitch bend to ± 2 semitones for high resolution tuning, and (3) A forward compatible prototype engine that complies with the emerging MIDI 2.0 standard. Both MTS and MPE modes provide resolution that is at least one order of magnitude finer than those deviation intervals present in equal temperament, which enables a perceptively transparent means of achieving adaptive tuning via native deployment in the browser.

1. INTRODUCTION

The study of ubiquitous music (ubimus) applies a social and community-based mosaic of methods to develop computer systems and devices for music composition and performance, human-computer interaction, and educational endeavours [1]. As such, recent ubimus frameworks have opened fresh avenues for the design of conceptual approaches and for the improvement of legacy techniques [2].

Recent ubimus research proposes struck-string interaction as a framework that fosters the use of generative techniques to explore the relationship between tuning and sonic qualities [4]. Su et al. [5] developed an initial toolkit for implementing Just Intonation in struck-string interaction, and demonstrated a prototype for converting midi data to JI. Within this framework, Su et al. [6] reviewed existing systems, identifying the need for a comprehensive, browser-based architecture to support both real-time tuning and access to external hardware support. Their work suggests further possibilities of application.

One area where struck-string interaction may foster aesthetic diversification is adaptive tuning. The assignment of specific frequencies to discrete pitches constitutes a field of application that has implications beyond the current equal temperament model (12-TET), adopted by most technology-based musical devices. This model divides the octave into twelve parts. As such, the intervals are approximations of pure integer frequency ratios. Just Intonation (JI) is an alternative method of tuning based on exact simple-integer frequency ratios – which are perceived as being the more consonant than equal-tempered intervals; however, there is no single fixed JI scale that can accommodate all key signatures [3]. Therefore, to deploy JI within tonal systems it is necessary to retune the scales when the harmonic context changes.

Handling microtonal pitch variations within computational ecosystems designed for fixed-pitch, equal-tempered systems tends to be problematic. Developers usually select either (1) the official MIDI Tuning Standard (MTS) — which employs System Exclusive (SysEx) messages to permit precise control — though very few consumer-grade hardware products utilise MTS natively [7]; or (2) Polyphonic Pitch Bend Methods — which function with a wide variety of hardware but provide limited resolution headroom and demand advanced channel management [8]. In browser-based contexts, security presents another caveat due to blocking of SysEx messages by default.

This paper presents Instant Harmonies, a unified architecture that addresses precision, compatibility, and security using a hierarchical fallback strategy. The system implements MTS for scale-based tuning when a SysEx permission is available, a precision-optimized MPE mode is proposed as a second-level fallback, and a proof-of-concept MIDI 2.0 engine provides support for forward compatibility. This browser-native approach, utilizing the Web MIDI API, allows for adaptive Just Intonation on any MIDI compatible keyboard instrument without requiring the installation of special software.

2. BACKGROUND

2.1 Related Work

The basis of Adaptive Just Intonation is psychoacoustic research: our sensitivity to "beats," or periodic amplitude changes resulting from two close partials, is what generally determines how we perceive mistuned intervals [9]. The fifth of equal temperament deviates from pure by roughly 2 cents, while the major third deviates by about 14 cents [9, 10]. Mohrlok [3] determined that for clean timbres the audible limit for retuning steps is about 3–4 cents. Therefore, we will set our design threshold at approximately 3 cents.

There are several algorithms used to develop real time adaptive tuning systems. Sethares [11] developed a real-time system in Max that uses gradient descent to minimise sensory dissonance. Code [12] created Groven.Max, which maps keyboard input to a 36-tone JI scale across three MIDI channels. Hermode Tuning [3], which is integrated into both Logic Pro and Cubase, adjusts the intervals of a composition towards just ratios but limits the possible shifts from the equal-tempered reference line to plus/minus 20 cents (plus/minus 30 cents for each pitch); however, it was specifically designed to avoid modulating the microtonality of the music. Stange et al. [13] formulated adaptive tuning as a spring network of linear equations, which generates MIDI output through the use of 15 individually pitch-bent channels. Volkov [14] developed Pivotuner, a VST3/AU plugin with a configurable Key Determiner algorithm and built-in MPE support.

Moussa [7] identified three common transportation methods of MIDI microtuning protocols — MTS, manufacturer-specific SysEx, and polyphonic pitch bend — and noted that although MTS provides a high resolution (0.0061 cents), its adoption rate has remained relatively low. Celik [8] created a real-time dynamic microtonal channel manager in Max/MSP. OddSound

MTS-ESP [15] provided a software-only solution that utilised inter-process communication to share tuning information among multiple processes within a DAW session. However, as with the majority of other systems, these systems were limited to running on desktop computers. The development of the Web MIDI API [16] enables browser-based MIDI interactions, although Baratè and Ludovico [17] note that it currently remains a W3C working draft and that support for it is limited to primarily Chromium-based browsers. Our architecture addresses these limitations by providing a hierarchical fallback mechanism between MTS and precision-optimised MPE, along with browser-native deployment, support for external hardware, and forward-compatible MIDI 2.0 support.

2.2 Just Intonation And Resolution Requirements

Nicholson and Sabat [18] explain the perceptual mechanisms that allow JI. Just Intonation is a method of tuning musical intervals as a small number of frequency ratios to create a unique periodic resonance. As the interval between two fundamentals gets closer to a simple ratio, the harmonic partials associated with each fundamental begin to align, and the “beating” effect is eliminated or reduced, thus isolating the sonic characteristics of the interval. As the ratios used become more complex, the fusion of the two sounds creates a periodic harshness, and eventually the ratios become so complex that the fusion is difficult to distinguish from tempered intervals [18].

We have established a conservative design threshold of about 3 cents based upon the smallest musically meaningful interval deviation (approximately 2 cents for the ET fifth) and the smallest audibly distinguishable tuning increment [3]. A system’s quantisation error should therefore be at least an order of magnitude smaller than the above value. Both operating modes of our system provide much greater resolution than the minimum required. The MTS Scale/Octave 2-Byte format provides a resolution of about 0.012 cents, based upon the 14-bit word length and the total number of discrete steps available:

$$R_{mts}^{cents} = 200 \text{ cents} / 2^{14} = 200 / 16384 \approx 0.0122 \text{ cents (1)}$$

The RPN-negotiated MPE mode we have developed, which restricts pitch-bend sensitivity to ± 2 semitones, provides a resolution of about 0.024 cents:

$$R^{mpe} = 400 \text{ cents} / 2^{14} = 400 / 16384 \approx 0.0244 \text{ cents (2)}$$

In both cases, these values are considerably smaller than our established design threshold of 3 cents. The standard MPE specification¹ allows for a pitch-bend range of ± 48 semitones on member channels. If we use the full 9600 cent range provided by the specification (which would yield a 14-bit resolution of $9600/16384 \approx 0.59$ cents per step), there would be very little margin left over before exceeding our established design threshold. However, our MPE mode uses only one quarter of the total pitch-bend range provided by the standard MPE specification, and yet it provides a 24× improvement in resolution while also being MPE compliant due to the RPN 0 parameter negotiation used on the member channels.

3. SYSTEM ARCHITECTURE

The implemented system utilises the Web MIDI API [16] to allow browser access to external MIDI devices. Upon activation of the proposed system, the first step is to verify if the user has granted SysEx permissions; this is a required safety measure for the transmission of MTS messages. Once SysEx permissions have been granted, the system will assume that the connected external device supports MTS and attempt to transmit tuning messages. Since MIDI 1.0 is unidirectional, there is no established method to determine if an external device supports MTS [19]; therefore, SysEx messages are classified as “fire and forget”. If a user finds that the MTS tuning mode does not yield the expected output, they may opt to manually select the MPE mode via the user interface. This option will then be saved as default for future usage.

The proposed permission-based architecture addresses both the trade-offs of compatibility vs. precision and provides a layer of security to the potential issue of the user transmitting any type of data to a device via the browser environment. By default, browser environments prohibit the transmission of SysEx messages since they could potentially be used to transmit any type of data to the connected device,

including but not limited to firmware updates or factory resets. The MPE mode serves as both a compatible and secure bridge by utilizing only Standard Channel Voice Messages that do not require special permissions.

3.1 Primary Method: MIDI Tuning Standard (MTS)

In key based adaptive Just Intonation (JI) tuning, the system determines the currently occurring tonal center in the music being played, and retunes the 12 pitch class of the chromatic scale to the pure ratio of Just Intonation as compared to this keynote [13]. Since JI is defined in terms of a particular keynote and therefore must be calculated again for every different key, the tuning must be recalculated on the basis of the new key whenever the key changes. A key based approach therefore differs from those methods which address the problem of minimising the perceived dissonance of a simultaneous set of pitches by changing the pitch of each sounding note independently [11].

The tonality-based approach will always tune the 12 pitches simultaneously, as this is a computationally efficient way to implement a scale. As such, an efficient MTS message to support this type of adaptive tuning is the Scale/Octave Tuning 2-Byte message (Sub-ID#2 = 0x09).² The Scale/Octave Tuning 2-Byte message retunes all 12 pitch classes (i.e. C through B) for all the selected MIDI channel(s), making this method suitable for adaptive tuning applications where the fundamental frequencies are aligned with the key changes. For each of the 12 pitch classes, the system computes the cent offset between the predefined 5-limit JI ratio and its equal-tempered counterpart, then encodes this offset as a 14-bit value in the MTS Scale/Octave 2-Byte message.

3.2 Fallback Method: Precision-Optimised MPE

When MTS is unavailable, MPE is used. We changed the pitch-bend sensitivity from the standard range (± 12 semitones) to ± 2 semitones with registered parameter number (RPN) 0, thus limiting the range of expression, but improving tuning accuracy to 0.0244 cents per step. As per the MPE specifications,³ the Manager Channel for the zone wide messages in Channel 1, while the Member Channels 2 through 16 are the ones that support individual notes. (The MPE specifications also

¹MIDI Manufacturers Association & Association of Musical Electronics Industry (2022), *MIDI Polyphonic Expression Specification* (M1-100-UM v1.1), available from <https://midi.org>.

²MIDI Manufacturers Association & Association of Musical Electronics Industry (2020), *MIDI Tuning Updated Specification*, documents CA-020 and CA-021, available from <https://midi.org/midi-tuning-updated-specification>.

³MIDI Manufacturers Association & Association of Musical Electronics Industry (2022), *MIDI Polyphonic Expression Specification* (M1-100-UM v1.1), available from <https://midi.org>.

allow for dual zones, with an upper zone being supported by channel 16; however, we are using a single lower zone.) RPN 0 was sent to all the member channels with data entry MSB at 2 semitones to prevent the sensitivity to be changed.

As MPE assigns a specific member channel to every sounding MIDI note, there is a polyphony limit imposed on the 15 channels available, this demands an allocation strategy. The method implemented keeps a pool of free member channels, alongside a queue that is ordered based upon the usage of the member channels to keep track of which note currently occupies each member channel. If a new note arrives, and there is a member channel available in the free pool, the next available member channel will be used; if all member channels are occupied then the member channel associated with the least recently activated note will be evicted. This is an eviction policy similar to cache replacement theory, and has been applied to the multi-channel pitch-bend allocation problem first described by Moussa [7]. The Least Recently Used (LRU) strategy ensures that the pitch bend is least likely to be perceptually relevant is the one that is evicted.

3.3 Proof-of-Concept: MIDI 2.0 Integration

We have developed a MIDI 2.0 Tuning Engine which generates Universal MIDI Packet (UMP) Structures. Since the WebMIDI API has no support for sending UMP messages [17], this module is currently running in File Export Mode. The Tuning Engine utilises the Registered Per-Note Controller #3; Pitch 7.25 as identified in the MIDI 2.0 Protocol Specification.⁴ Pitch 7.25 will utilise a 32 bit value (7 bits to represent the Semitone Integer, 25 bits to represent the Fractional Part). This yields a theoretical resolution of 0.000003 Cents — roughly one million times finer than the perceptual limit.

4. COMPARATIVE EVALUATION

To validate our design, we compared the effective resolution of each tuning protocol to the human beat-detection threshold. We applied a conservative 3-cent margin of error, as outlined by Mohrlök [3], informed by Vos [9]. The Perceptual Margin column in Table 1 provides the resolution as a multiple of the 3-cent margin, i.e. a multiple of 123 $\times$ indicates the resolution is 123 times finer than the margin of error.

⁴MIDI Manufacturers Association & Association of Musical Electronics Industry (2023), *Universal MIDI Packet (UMP) Format and MIDI 2.0 Protocol* (M2-104-UM v1.1.2), available from <https://midi.org>.

Figure 1 displays the resolution of the two tuning methods, on a logarithmic scale.

Table 1: Resolution Analysis of the Proposed Tuning Methods

Method	Bit Depth	Range	Resolution (cents)	Perceptual Margin
MPE (Standard)	14 bit	± 48 semitones	~ 0.59	$\sim 5\times$
MPE (Optimised)	14 bit	± 2 semitones	0.0244	$\sim 123\times$
MTS (Scale/Oct)	14 bit	± 1 semitone	0.0122	$\sim 246\times$
MIDI 2.0	32 bit	Per-note absolute	0.000003	$\sim 1,000,000\times$

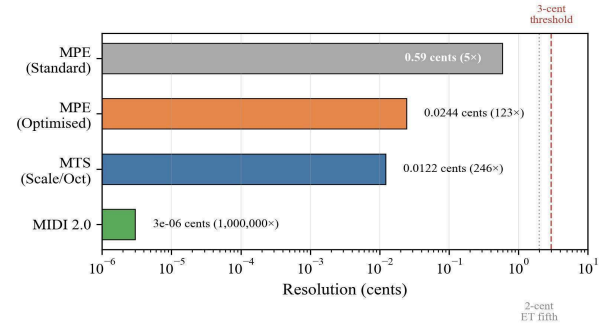


Figure 1: Logarithmic comparison of tuning resolution across MIDI protocols. The 3-cent perceptual threshold (dashed red line) and the 2-cent ET fifth deviation (dotted grey line) are shown for reference. All proposed modes substantially exceed the threshold.

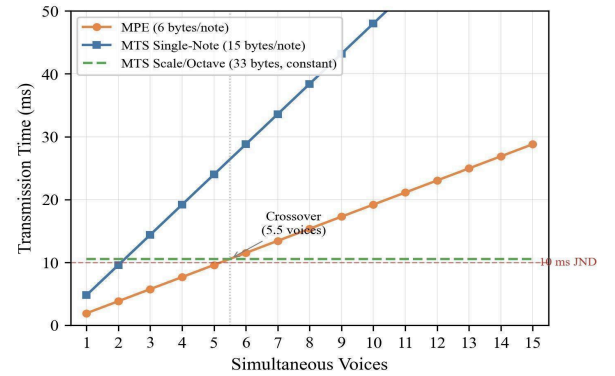


Figure 2: Transmission time as a function of simultaneous voices. MPE (6 bytes/note) has lower overhead for sparse textures, while MTS Scale/Octave

(33 bytes, constant) becomes more efficient beyond 5.5 voices. MTS Single-Note (15 bytes/note) scales linearly. The 10 ms percussive JND is shown for reference [20].

The transmission structure and time for each tuning protocol is provided in Table 2. Both MTS and MPE have resolutions exceeding the human beat-detection threshold. However, there is a significant cost associated with bandwidth and latency. For example, the baud rate for serial MIDI 1.0 is 31,250 bits per second.⁵ Given an effective throughput of 3,125 bytes per second (meaning each byte takes 0.32 milliseconds).

Table 2: MIDI Message Byte Counts and Transmission Times

Message Type	Structure	Bytes	Time (ms)	Source
MTS Scale/Octave 2-Byte	F0 7F id 08 09 ff gg hh [ss tt]×12 F7	33	10.56	CA-021 ²
MTS Single-Note	F0 7F id 08 02 tt ll [kk xx yy zz] F7	12	3.84	CA-020 ²
MPE Pitch Bend	En ll mm	3	0.96	MIDI 1.0 ⁴
Note On/Off	9n/8n kk vv	3	0.96	MIDI 1.0 ⁴

There are clear architectural trade-offs between the two tuning protocols. First, MTS Scale/Octave has constant time-scale retuning (a single 33-byte message retunes all 12 pitch-classes regardless of how many events are being played, making it ideal for use in JI applications that rely on tonality-based adaptive methods). Second, MPE’s per-note granularity features lower latency in sparse textures and the ability to tune a single note without affecting other pitches. However, as illustrated in Figure 2, the transmission time increases linearly with polyphony.

5. DISCUSSION

Both modes achieve a tuning resolution finer than those specified in perceptual thresholds. We will examine three architectural aspects of the system and will point to three limitations.

Hierarchical fallback.

Moussa [7] established the limits of single-port channel assignment and introduced an overall routing method that makes use of three different physical MIDI output channels: one for instruments that can generate

polyphonic aftertouch, one for MTS capable instruments and one for instruments that do not have either of these features. Similar to Moussa’s system, our system allows for graceful degradation through software level protocol-switching in a single browser session using SysEx permissions as the fallback criteria. This adds a security aspect to desktop-based systems, as browsers block SysEx by default. Therefore, the precision-optimised MPE mode in our system functions as a compatibility bridge and a security compliance alternative at the same time. As illustrated previously, both modes far surpass the conservative 3-cent threshold by more than two orders of magnitude. Therefore, this software-based fallback may supplement hardware level flexibility and reconfigurability such as described by Keller et al. in their ubimus plugging framework based on FPGA modules [4].

Browser-Native Deployment

While several previous systems supported external MIDI hardware — including Stange et al. [13], Celik [8], Code [12], and Moussa [7] — all of them require commercial applications or specialised hardware to accomplish this task. Our system supports hardware output via the Web MIDI API, allowing browser applications to interact with complex MIDI setups that may include disparate devices [17]. The zero-installation model employed in our system is aligned with the ubimus objective of promoting creative participation among diverse stakeholders [1].

Limitations

Three technical limitations need to be acknowledged. Firstly, the unidirectional nature of MIDI 1.0 does not provide for automatic identification of MTS capabilities [19]. Secondly, our MPE mode relies on synthesisers correctly identifying and responding to RPN 0 messages; if synthesisers ignore non-default sensitivity values then the resolution decreases from 0.024 to about 0.59 cents. Lastly, there may also be timing jitters introduced due to the USB-MIDI interface. The fact that the Web MIDI API is currently in working draft status, limited to Chromium-based browsers, dependent on MIDI hardware being present may create usability challenges for non-expert users in configuring MIDI settings [17].

Future work will focus on addressing the following areas: (1) provide support for real-time MIDI 2.0 UMP transmissions for an enhanced Web MIDI API, thereby eliminating the 15 voice polyphony limit and the overhead associated with allocating MPE channels; (2) incorporate symbolic representations (such as MusicXML) to avoid the latency associated with

⁵MIDI Manufacturers Association & Association of Musical Electronics Industry (1996), *The Complete MIDI 1.0 Detailed Specification*, Document Version 96.1, available from <https://midi.org>.

key-identification algorithms; and (3) employ MIDI 2.0 Capability Inquiry (MIDI-CI)⁶ to automatically negotiate the protocol, yielding an entirely automated device-discovery process.

6. CONCLUSION

In this paper, we have documented a multi-modal MIDI-based architecture that addresses the longstanding trade-off between tuning accuracy and hardware compatibility in adaptive Just Intonation. The system employs a hierarchical fallback strategy. MTS Scale/Octave Tuning is utilised when SysEx permissions are available, and a precision-optimised MPE mode — which locks pitch bend sensitivity to ± 2 semitones by RPN 0 negotiations — is used as a fallback if MTS is either unavailable or ineffective. A proof-of-concept MIDI 2.0 engine ensures forward compatibility with future standards.

Both operational modes surpass perceptual requirements: MTS provides an approximate 0.012 cent resolution (ca. 246 times a conservative 3-cent threshold); Precision Optimized MPE furnishes a 0.024-cent resolution (approximately 123 times the threshold). The proposed contribution is architectural rather than algorithmic: to the best of our knowledge, this is the first adaptive tuning system to furnish a hierarchical fallback mechanism between standard MIDI tuning protocols, featuring direct external hardware output from a zero-installation web-browser environment. As part of the ubimus research agenda on struck-string interaction [4, 5, 6], our system enables adaptive Just Intonation without specialised hardware or software requirements.

7. ACKNOWLEDGEMENTS

This work is part of the “Instant Harmonies” project, a doctoral research project at Maynooth University that aims to make Just Intonation tools easily accessible and adaptable. The goal of the project is to make microtonal music-making more accessible by creating a free, browser-based platform that lets both professional musicians and amateurs experience Just Intonation on their existing MIDI devices without specialised hardware or software.

8. REFERENCES

- [1] D. Keller, V. Lazzarini, and M. S. Pimenta, *Ubiquitous Music*. Berlin and Heidelberg: Springer International Publishing, 2014, doi: 10.1007/978-3-319-11152-0.
- [2] D. Keller, F. Schiavoni, and V. Lazzarini, “Ubiquitous music: Perspectives and challenges,” *J. New Music Res.*, vol. 48, no. 4, pp. 309–315, 2019, doi: 10.1080/09298215.2019.1659828.
- [3] W. Mohrlok, “The Hermode Tuning system,” 2003. [Online]. Available: <https://sethars.engr.wisc.edu/paperspdf/hermode.pdf>
- [4] D. Keller, A. Jagwani, and V. Lazzarini, “The ubimus plugging framework: Deploying FPGA-based prototypes for ubiquitous music hardware design,” *Computers*, vol. 14, no. 4, p. 155, 2025, doi: 10.3390/computers14040155.
- [5] R. Su, J. Timoney, and D. Keller, “Just intonation and inharmonicity in struck-string interaction,” in *Proc. Ubiquitous Music Symp. (UbiMus 2024)*, Macau, China, 2024.
- [6] R. Su, J. Timoney, and D. Keller, “MIDI adaptive tuning strategies by means of AI-based struck-string interaction in ubimus,” in *Proc. Ubiquitous Music Symp. (UbiMus 2025)*, 2025.
- [7] A. S. Moussa, “Perception-based microtuning over MIDI networks,” *IEEE Multimedia*, vol. 13, no. 1, pp. 56–64, Jan.–Mar. 2006, doi: 10.1109/MMUL.2006.18.
- [8] S. Celik, “Micro-MIDI: A real time, dynamic microtonal MIDI application,” *Int. J. Eng. Sci. Invention*, vol. 5, no. 9, pp. 1–7, 2016.
- [9] J. Vos, “The perception of pure and mistuned musical fifths and major thirds: Thresholds for discrimination, beats, and identification,” *Perception & Psychophysics*, vol. 32, no. 4, pp. 297–313, 1982, doi: 10.3758/BF03206236.
- [10] S. Schwär, S. Rosenzweig, and M. Müller, “A differentiable cost measure for intonation processing in polyphonic music,” in *Proc. 22nd ISMIR Conf.*, 2021, pp. 626–633.
- [11] W. A. Sethares, “Real-time adaptive tunings using Max,” *J. New Music Res.*, vol. 31, no. 4, pp. 347–355, 2002, doi: 10.1076/jnmr.31.4.347.14163.
- [12] D. L. Code, “Groven.Max: An adaptive tuning system for MIDI pianos,” *Comput. Music J.*, vol. 26, no. 2, pp. 50–61, 2002, doi: 10.1162/014892602760137185.
- [13] K. Stange, C. Wick, and H. Hinrichsen, “Playing music in just intonation: A dynamically adaptive tuning scheme,” *Comput. Music J.*, vol. 42, no. 3, pp. 47–62, 2018, doi: 10.1162/comj_a_00478.
- [14] D. Volkov, “Pivotuner: Automatic real-time pure intonation and microtonal modulation,” in *Proc. Int. Comput. Music Conf. (ICMC)*, Shenzhen, China, 2023.
- [15] O. Cash, D. Gamble, and D. Hancock, “MTS-ESP Mini user guide,” ver. 1.08, OddSound, 2021.
- [16] C. Wilson and J. Kalliokoski, “Web MIDI API,” W3C Working Draft, 2015. [Online]. Available: <https://www.w3.org/TR/webmidi/>

⁶MIDI Manufacturers Association & Association of Musical Electronics Industry (2024), *MIDI-CI Profile for MIDI Polyphonic Expression* (M2-120-UM v2.0.3), available from <https://midi.org>.

- [17] A. Baratè and L. A. Ludovico, “Web MIDI API: State of the art and future perspectives,” *J. Audio Eng. Soc.*, vol. 70, no. 11, pp. 918–925, 2022, doi: 10.17743/jaes.2022.0028.
- [18] T. Nicholson and M. Sabat, “Fundamental principles of just intonation and microtonal composition,” Universität der Künste Berlin, Studio für Intonationsforschung und mikrotonale Komposition, 2018.
- [19] S. Luke and L. A. Ludovico, “MIDI 1.5, so to speak,” in *Proc. Sound Music Comput. Conf. (SMC 2024)*, 2024, pp. 373–382.
- [20] Jack, Robert & Mehrabi, Adib & Stockman, Tony & McPherson, Andrew. “Action-sound Latency and the Perceived Quality of Digital Musical Instruments: Comparing Professional Percussionists and Amateur Musicians.” *Music Perception* 36 (1): 109–128, 2018. (DOI: 10.1525/mp.2018.36.1.109)